

6.4.1 Inverse Quadratic Interpolation

Inverse quadratic interpolation is similar in spirit to the secant method. As in Fig. 6.8a, the secant method is based on computing a straight line that goes through two guesses. The intersection of this straight line with the x axis represents the new root estimate. For this reason, it is sometimes referred to as a *linear interpolation method*.

Now suppose that we had three points. In that case, we could determine a quadratic function of x that goes through the three points (Fig. 6.8b). Just as with the linear secant method, the intersection of this parabola with the x axis would represent the new root estimate. And as illustrated in Fig. 6.8b, using a curve rather than a straight line often yields a better estimate.

Although this would seem to represent a great improvement, the approach has a fundamental flaw: it is possible that the parabola might not intersect the x axis! Such would be the case when the resulting parabola had complex roots. This is illustrated by the parabola, $y = f(x)$, in Fig. 6.9.

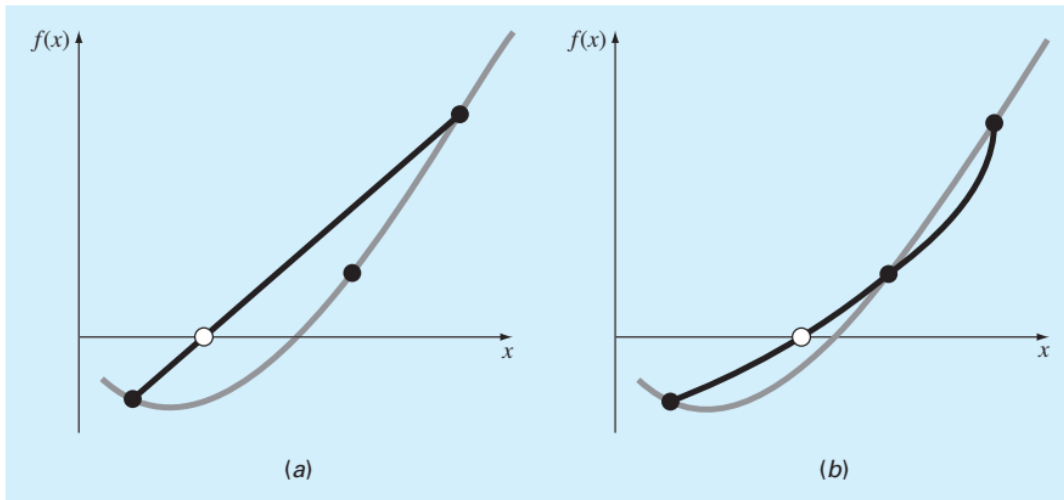
The difficulty can be rectified by employing inverse quadratic interpolation. That is, rather than using a parabola in x , we can fit the points with a parabola in y . This amounts to reversing the axes and creating a “sideways” parabola [the curve, $x = f(y)$, in Fig. 6.9].

If the three points are designated as (x_{i-2}, y_{i-2}) , (x_{i-1}, y_{i-1}) , and (x_i, y_i) , a quadratic function of y that passes through the points can be generated as

$$g(y) = \frac{(y - y_{i-1})(y - y_i)}{(y_{i-2} - y_{i-1})(y_{i-2} - y_i)}x_{i-2} + \frac{(y - y_{i-2})(y - y_i)}{(y_{i-1} - y_{i-2})(y_{i-1} - y_i)}x_{i-1} + \frac{(y - y_{i-2})(y - y_{i-1})}{(y_i - y_{i-2})(y_i - y_{i-1})}x_i \quad (6.10)$$

FIGURE 6.8

Comparison of (a) the secant method and (b) inverse quadratic interpolation. Note that the approach in (b) is called “inverse” because the quadratic function is written in y rather than in x .



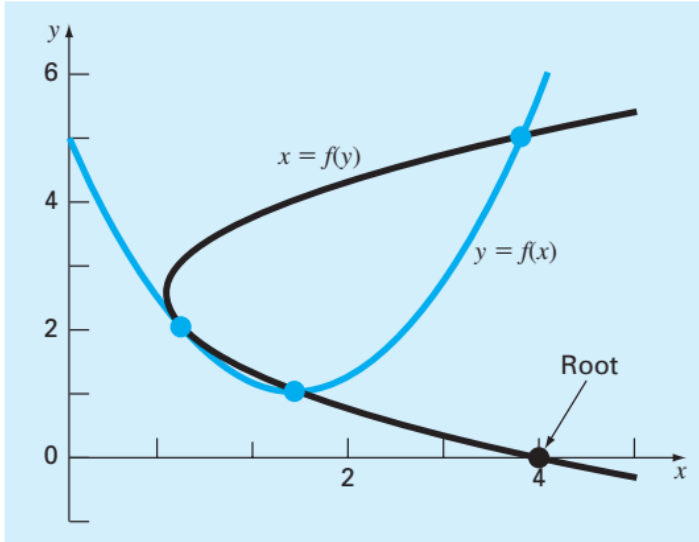


FIGURE 6.9

Two parabolas fit to three points. The parabola written as a function of x , $y = f(x)$, has complex roots and hence does not intersect the x axis. In contrast, if the variables are reversed, and the parabola developed as $x = f(y)$, the function does intersect the x axis.

As we will learn in Sec. 18.2, this form is called a *Lagrange polynomial*. The root, x_{i+1} , corresponds to $y = 0$, which when substituted into Eq. (6.10) yields

$$x_{i+1} = \frac{y_{i-1}y_i}{(y_{i-2} - y_{i-1})(y_{i-2} - y_i)}x_{i-2} + \frac{y_{i-2}y_i}{(y_{i-1} - y_{i-2})(y_{i-1} - y_i)}x_{i-1} + \frac{y_{i-2}y_{i-1}}{(y_i - y_{i-2})(y_i - y_{i-1})}x_i \quad (6.11)$$

As shown in Fig. 6.9, such a “sideways” parabola always intersects the x axis.

Problem is given: $w = 10 \text{ N/m}$, $y_0 = 5 \text{ m}$, and $y = 15 \text{ m}$.

6.17 A catenary cable is one which is hung between two points not in the same vertical line. As depicted in Fig. P6.17a, it is subject to no loads other than its own weight. Thus, its weight acts as a uniform load per unit length along the cable w (N/m). A free-body diagram of a section AB is depicted in Fig. P6.17b, where T_A and T_B are the tension forces at the end. Based on horizontal and vertical force balances, the following differential equation model of the cable can be derived:

$$\frac{d^2y}{dx^2} = \frac{w}{T_A} \sqrt{1 + \left(\frac{dy}{dx}\right)^2}$$

Calculus can be employed to solve this equation for the height of the cable y as a function of distance x :

$$y = \frac{T_A}{w} \cosh\left(\frac{w}{T_A}x\right) + y_0 - \frac{T_A}{w}$$

- (a) Use a numerical method to calculate a value for the parameter T_A given values for the parameters $w = 10$ and $y_0 = 5$, such that the cable has a height of $y = 15$ at $x = 50$.
- (b) Develop a plot of y versus x for $x = -50$ to 100 .

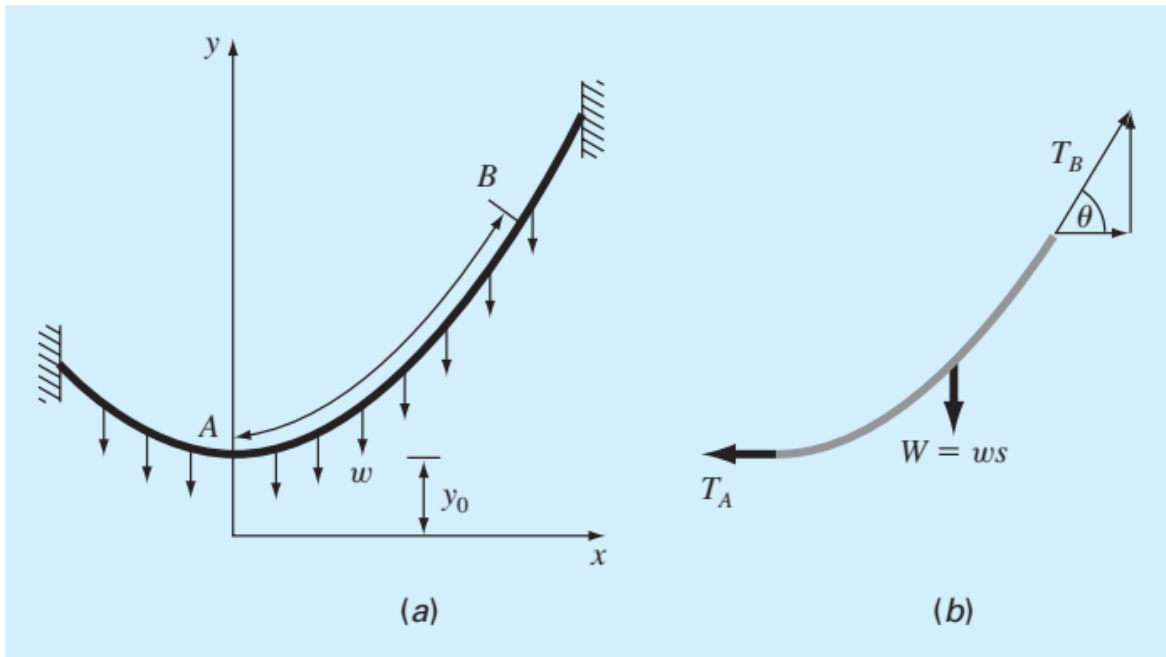


FIGURE P6.17

Solve the problem using inverse quadratic interpolation method, and compare it with newton Raphson method. Draw appropriate graphs as required.

If the method fails comment on that.

What technique/ algorithm was used for updating the guess during iterations.